arXiv:0705.2011v1 [cs.AI] 14 May 2007

Multi-Dimensional Recurrent Neural Networks

Alex Graves, Santiago Fernández, Jürgen Schmidhuber
IDSIA
Galleria 2, 6928 Manno, Switzerland
{alex,santiago,juergen}@idsia.ch

February 11, 2013

Abstract

Recurrent neural networks (RNNs) have proved effective at one dimensional sequence learning tasks, such as speech and online handwriting recognition. Some of the properties that make RNNs suitable for such tasks, for example robustness to input warping, and the ability to access contextual information, are also desirable in multidimensional domains. However, there has so far been no direct way of applying RNNs to data with more than one spatio-temporal dimension. This paper introduces multi-dimensional recurrent neural networks (MDRNNs), thereby extending the potential applicability of RNNs to vision, video processing, medical imaging and many other areas, while avoiding the scaling problems that have plagued other multi-dimensional models. Experimental results are provided for two image segmentation tasks.

1 Introduction

Recurrent neural networks (RNNs) were originally developed as a way of extending neural networks to sequential data. Because of their recurrent connections, RNNs are able to make use of previous context. Moreover, because RNNs can adapt to stretched or compressed input patterns by varying the rate of change of their internal state, they are more robust to temporal warping than non-recursive models.

In recent experiments, RNNs have outperformed hidden Markov Models (HMMs) in a variety of speech and online handwriting recognition tasks [5, 4, 3, 13].

Access to contextual information and robustness to warping are also important when dealing with multi-dimensional data. For example, a face recognition algorithm should be able to access the entire face at once, and it should be robust to changes in perspective, distance etc. It therefore seems desirable to apply RNNs to such tasks.

However, the RNN architectures used so far have been explicitly one dimensional, meaning that in order to use them for multi-dimensional tasks, the data must be pre-processed to one dimension, for example by presenting one vertical line of an image at a time to the network. The most successful use of neural networks for multi-dimensional data has been the application of convolution networks to image processing tasks such as digit recognition [11, 15]. One disadvantage of convolution nets is that because they are not recurrent, they rely on hand specified kernel sizes to introduce context. Another disadvantage is that they don't scale well to large images. For example, sequences of handwritten digits must be pre-segmented into individual characters before they can be recognised by convolution nets [11].

multi-dimensional 主要在于对于image的conv

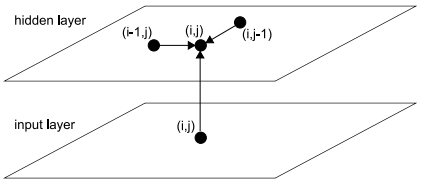

Figure 1: 2D RNN Forward pass.

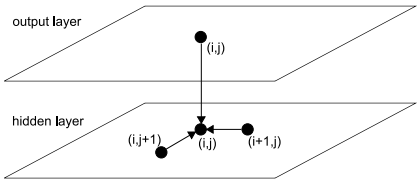

Figure 2: 2D RNN Backward pass.

Various statistical models have been proposed for multi-dimensional data, notably multi-dimensional HMMs. However, multi-dimensional HMMs suffer from two severe drawbacks: (1) the time required to run the Viterbi algorithm, and thereby calculate the optimal state sequences, grows exponentially with the number of data points; (2) the number of transition probabilities, and hence the required memory, grows exponentially with the data dimensionality. Numerous approximate methods have been proposed to alleviate one or both of these problems, including pseudo 2D and 3D HMMs [8], isolating elements [12], approximate Viterbi algorithms [10], and dependency tree HMMs [9]. However, none of these methods are able to exploit the full multi-dimensional structure of the data.

As we will see, multi dimensional recurrent neural networks (MDRNNs) bring the benefits of RNNs to multi-dimensional data, without suffering from the scaling problems described above.

Section 2 describes the MDRNN architecture, Section 3 presents two experiments on image segmentation, and concluding remarks are given in Section 4.

2 Multi-Dimensional Recurrent Neural Networks

The basic idea of MDRNNs is to replace the single recurrent connection found in standard RNNs with as many recurrent connections as there are dimensions in the data. During the forward pass, at each point in the data sequence, the hidden layer of the network receives both an external input and its own activations from one step back along all dimensions. Figure 1 illustrates the two dimensional case.

Note that, although the word *sequence* usually connotes one dimensional data, we will use it to refer to data examplars of any dimensionality. For example, an image is a two dimensional sequence, a video is a three dimensional sequence, and a series of fMRI brain scans is a four dimensional sequence.

Clearly, the data must be processed in such a way that when the network reaches a point in an n-dimensional sequence, it has already passed through all the points from which it will receive its previous activations. This can be ensured by following a suitable ordering on the points $\{(x_1, x_2, ..., x_n)\}$. One example of a suitable ordering is $(x_1, \ldots, x_n) < (x'_1, \ldots, x'_n)$ if $\exists\ m \in (1, \ldots, n)$ such that $x_m < x'_m$ and $x_i = x'_i\ \forall\ i \in (1, \ldots, m-1)$. Note that this is not the only possible ordering, and that its realisation for a particular sequence depends on an arbitrary choice of axes. We will return to this point in Section 2.1. Figure 3 illustrates the ordering for a 2 dimensional sequence.

The forward pass of an MDRNN can then be carried out by feeding forward the input and the n previous hidden layer activations at each point in the ordered input sequence, and storing the resulting hidden layer activations. Care must be taken at the sequence boundaries not to feed forward activations from points outside the sequence.

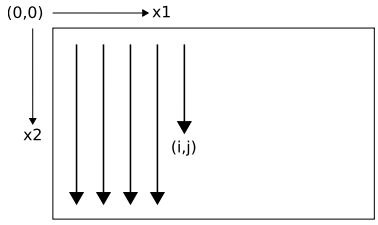

Figure 3: 2D sequence ordering. The MDRNN forward pass starts at the origin and follows the direction of the arrows. The point (i,j) is never reached before both (i-1,j) and (i,j-1).

Note that each 'point' in the input sequence will in general be a multivalued vector. For example, in a two dimensional colour image, the inputs could be single pixels represented by RGB triples, or blocks of pixels, or the outputs of a preprocessing method such as a discrete cosine transform.

The error gradient of an MDRNN (that is, the derivative of some objective function with respect to the network weights) can be calculated with an n-dimensional extension of the backpropagation through time (BPTT [16]) algorithm. As with one dimensional BPTT, the sequence is processed in the reverse order of the forward pass. At each timestep, the hidden layer receives both the output error derivatives and its own n 'future' derivatives. Figure 2 illustrates the BPTT backward pass for two dimensions. Again, care must be taken at the sequence boundaries.

At a point $\mathbf{x} = (x_1, \ldots, x_n)$ in an n-dimensional sequence, define $i_j^{\mathbf{x}}$ and $h_k^{\mathbf{x}}$ respectively as the activations of the jth input unit and the kth hidden unit. Define w_{kj} as the weight of the connection going from unit j to unit k. Then for an n-dimensional MDRNN whose hidden layer consists of summation units with the $tanh$ activation function, the forward pass for a sequence with dimensions $(X_1, X_2, \ldots, X_n)$ can be summarised as follows:

for $x_1 = 0$ to $X_1 - 1$ **do**
  for $x_2 = 0$ to $X_2 - 1$ **do**
    ...
    for $x_n = 0$ to $X_n - 1$ **do**
      initialize $a \leftarrow \sum_j in_j^{\mathbf{x}} w_{kj}$
      for $i = 1$ to n **do**
        if $x_i > 0$ **then**
          $a \leftarrow a + \sum_j h_j^{(x_1,\ldots,x_i-1,\ldots,x_n)} w_{kj}$
      $h_k^{\mathbf{x}} \leftarrow tanh(a)$

Algorithm 1: MDRNN Forward Pass

Defining $\hat{o}_j^{\mathbf{x}}$ and $\hat{h}_k^{\mathbf{x}}$ respectively as the derivatives of the objective function with respect to the activations of the jth output unit and the kth hidden unit at point $\mathbf{x}$, the backward pass is:

Since the forward and backward pass require one pass each through the data sequence, the overall complexity of MDRNN training is linear in the number of data points and the number of network weights.

for $x_1 = X_1 - 1$ to 0 **do**
  for $x_2 = X_2 - 1$ to 0 **do**
    $\ldots$
    for $x_n = X_n - 1$ to 0 **do**
      initialize $e \leftarrow \sum_j \hat{o}_j^{\mathbf{x}} w_{jk}$
      for $i = 1$ to n **do**
        if $x_i < X_i - 1$ **then**
          $e \leftarrow e + \sum_j \hat{h}_j^{(x_1,\ldots,x_i+1,\ldots,x_n)} w_{jk}$
      $\hat{h}_k^{\mathbf{x}} \leftarrow tanh'(e)$

Algorithm 2: MDRNN Backward Pass

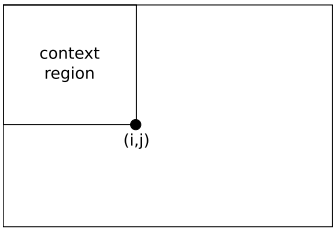

Figure 4: Context available at (i,j) to a 2D RNN with a single hidden layer.

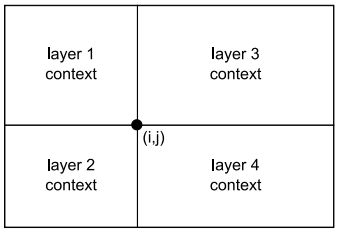

Figure 5: Context available at (i,j) to a multi-directional 2D RNN.

2.1 Multi-directional MDRNNs

At a point $(x_1, ..., x_n)$ in the input sequence, the network described above has access to all points $(x'_1, ..., x'_n)$ such that $x'_i \leq x_i \forall\, i \in (1, ..., n)$. This defines an n-dimensional 'context region' of the full sequence, as shown in Figure 4. For some tasks, such as object recognition, this would in principal be sufficient. The network could process the image as usual, and output the object label at a point when the object to be recognized is entirely contained in the context region.

Intuitively however, we would prefer the network to have access to the surrounding context in all directions. This is particularly true for tasks where precise localization is required, such as image segmentation.

For one dimensional RNNs, the problem of multi-directional context was solved in 1997 by the introduction of bidirectional recurrent neural networks (BRNNs) [14]. BRNNs contain two separate hidden layers that process the input sequence in the forward and reverse directions. The two hidden layers are connected to a single output layer, thereby providing the network with access to both past and future context.

BRNNs can be extended to n-dimensional data by using 2^n separate hidden layers, each of which processes the sequence using the ordering defined above, but with a different choice of axes. More specifically, the axes are chosen so that their origins lie on the 2^n vertices of the sequence. The 2 dimensional case is illustrated in Figure 6. As before, the hidden layers are connected to a single output layer, which now has access to all surrounding context (see Figure 5).

If the size of the hidden layers is held constant, multi-directional MDRNNs scales as $O(2^n)$ for n-dimensional data. In practice however, we have found that using 2^n small layers gives better results than 1 large layer with the same overall number of weights, presumably because the data processing is shared between the hidden layers. This also holds in one dimension, as previous experiments have demonstrated [5]. In

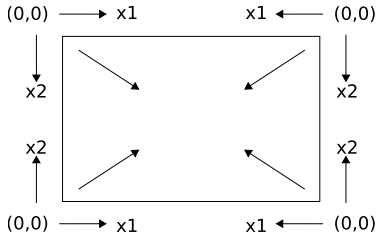

Figure 6: Axes used by the 4 hidden layers in a multi-directional 2D network. The arrows inside the rectangle indicate the direction of propagation during the forward pass.

any case the complexity of the algorithm remains linear in the number of data points and the number of parameters, and the number of parameters is independent of the data dimensionality.

For a multi-directional MDRNN, the forward and backward passes through an n-dimensional sequence can be summarised as follows:

1: For each of the 2^n hidden layers choose a distinct vertex of the sequence, then define a set of axes such that the vertex is the origin and all sequence co-ordinates are ≥ 0
2: Repeat Algorithm 1 for each hidden layer
3: At each point in the sequence, feed forward all hidden layers to the output layer

Algorithm 3: Multi-directional MDRNN Forward Pass

1: At each point in the sequence, calculate the derivative of the objective function with respect to the activations of output layer
2: With the same axes as above, repeat Algorithm 2 for each hidden layer

Algorithm 4: Multi-directional MDRNN Backward Pass

2.2 Multi-dimensional Long Short-Term Memory

So far we have implicitly assumed that the network can make use of all context to which it has access. For standard RNN architectures however, the range of context that can practically be used is limited. The problem is that the influence of a given input on the hidden layer, and therefore on the network output, either decays or blows up exponentially as it cycles around the network's recurrent connections. This is usually referred to as the *vanishing gradient problem* [6].

Long Short-Term Memory (LSTM) [7, 2] is an RNN architecture specifically designed to address the vanishing gradient problem. An LSTM hidden layer consists of multiple recurrently connected subnets, known as memory blocks. Each block contains a set of internal units, known as cells, whose activation is controlled by three multiplicative units: the input gate, forget gate and output gate. The effect of the gates is to allow the cells to store and access information over long periods of time, thereby avoiding the vanishing gradient problem.

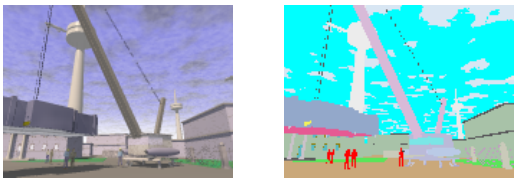

Figure 7: Frame from the Air Freight database, showing the original image (left) and the colour-coded texture segmentation (right).

The standard formulation of LSTM is explicitly one-dimensional, since the cell contains a single self connection, whose activation is controlled by a single forget gate. However we can easily extend this to n dimensions by using instead n self connections (one for each of the cell's previous states along every dimension) with n forget gates.

3 Experiments

3.1 Air Freight Data

The Air Freight database is a ray-traced colour image sequence that comes with a ground truth segmentation based on textural characteristics (Figure 7). The sequence is 455 frames long and contains 155 distinct textures. Each frame is 120 pixels high and 160 pixels wide.

The advantage of ray-traced data is the true segmentation can be defined directly from the 3D models. Although the images are not real, they are realistic in the sense that they have significant lighting, specular effects etc.

We used the sequence to define a 2D image segmentation task, where the aim was to assign each pixel in the input data to the correct texture class. We divided the data at random into a 250 frame train set, a 150 frame test set and a 55 frame validation set. Note that we could have instead defined a 3D task where the network processed the entire video as one sequence. However, this would have left us with only one exemplar.

For this task we used a multi-directional MDRNN with 4 LSTM hidden layers. Each layer consisted of 25 memory blocks, each containing 1 cell, 2 forget gates, 1 input gate, 1 output gate and 5 peephole weights. This gave a total 600 hidden units. The input and output activation functions of the cells were both tanh, and the activation function for the gates was the logistic sigmoid in the range $[0, 1]$. The input layer was size 3 (one each for the red, green and blue components of the pixels) and the output layer was size 155 (one unit for each textural class). The network contained 43,257 trainable weights in total. As is standard for classification tasks, the softmax activation function was used at the output layer, with the cross-entropy objective function [1]. The network was trained using online gradient descent (weight updates after every training sequence) with a learning rate of 10^{-6} and a momentum of 0.9.

The final pixel classification error rate was $7.3\,\%$ on the test set.

Figure 8: MNIST image before and after deformation.

Table 1: Image error rates on MNIST (pixel error rates in brackets)

Algorithm	Clean Test Set	Warped Test Set
MDRNN	1.1 % (0.5 %)	6.8 % (3.8 %)
Convolution	0.9 %	11.3 %

3.2 MNIST Data

The MNIST database [11] of isolated handwritten digits is a subset of a larger database available from NIST. It consists of size-normalized, centered images, each of which is 28 pixels high and 28 pixels wide and contains a single handwritten digit. The data comes divided into a training set with 60,000 images and a test set with 10,000 images. We used 10,000 of the training images for validation, leaving 50,000 for training.

The usual task on MNIST is to label the images with the corresponding digits. This is benchmark task for which many algorithms have been evaluated.

We carried out a slightly modified task where each pixel was classified according to the digit it belonged to, with an additional class for background pixels. However, the original task can be recovered by simply choosing the digit whose corresponding output unit had the highest cumulative activation for the entire sequence.

To test the network's robustness to input warping, we also evaluated it on an altered version of the MNIST test set, where elastic deformations had been applied to every image (Figure 8).

We compared our results with the convolution neural network that has achieved the best results so far on MNIST [15]. Note that we re-implemented the convolution network ourselves, and we did not augment the training set with elastic distortions, which gives a substantial improvement in performance.

The MDRNN for this task was identical to that for the Air Freight task with the following exceptions: the sizes of the input and output layers were now 1 (for grayscale pixels) and 11 (one for each digit, plus background) respectively, giving 27,511 weights in total, and the gradient descent learning rate was 10^{-5}.

For the distorted test set, we used the same degree of elastic deformation used by Simard [15] to augment the training set ($\sigma = 4.0$, $\alpha = 34.0$), with a different initial random field for every sample image.

Table 1 shows thatlthough the MDRNN performed slightly worse on the clean test set, its performance was considerably better on the warped test set. This suggests that MDRNNs are more robust to input warping than convolution networks.

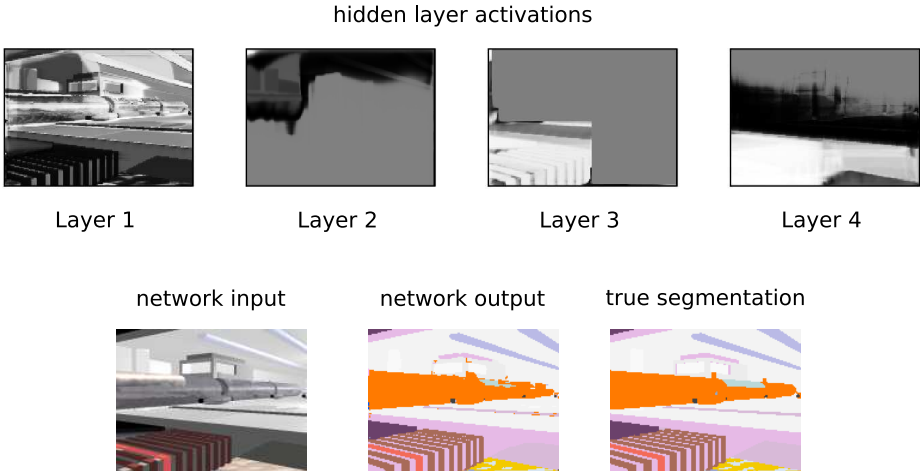

Figure 9: 2D MDRNN applied to an image from the Air Freight database. The hidden layer activations display one unit from each of the layers. A common behaviour is to 'mask off' parts of the image, exhibited here by layers 2 and 3.

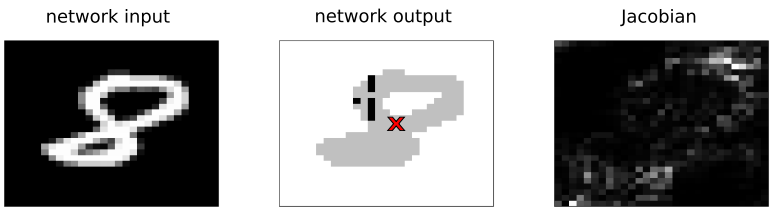

Figure 10: Jacobian matrix of a 2D RNN for an image from the MNIST database. The white outputs correspond to the class 'background' and the light grey ones to '8'. The black outputs represent misclassifications. The output pixel for which the Jacobian is calculated is marked with a cross. Absolute values are plotted for the Jacobian, and lighter colours are used for higher values.

3.3 Analysis

One benefit of two dimensional tasks is that the operation of the network can be easily visualised. Figure 9 shows the network activations during a frames from the Air Freight database. As can be seen, the network segments this image almost perfectly, in spite of difficult, reflective surfaces such as the glass and metal tube running from left to right. Clearly, classifying individual pixels in such a surface requires considerable use of context.

A precise measure of the network's sensitivity to context can be found by analysing the derivatives of the network outputs at a particular point $\mathbf{x}$ in the sequence with respect to the inputs at all points $\mathbf{x}'$ in the sequence. The matrix $\frac{\partial o_k^{\mathbf{x}}}{\partial in_j^{\mathbf{x}'}}$ of these derivatives is referred to as the *Jacobian* matrix. Figure 10 shows the absolute value of the Jacobian matrix for a single output during classification of an image from the MNIST database. It can be seen that the network responds to context from across the entire image, and

seems particularly attuned to the outline of the digit.

4 Conclusion

We have introduced multi-dimensional recurrent neural networks (MDRNNs), thereby extending the applicabilty of RNNs to n-dimensional data. We have added multi-directional hidden layers that provide the network with access to all contextual information, and we have developed a multi-dimensional variant of the Long Short-Term Memory RNN architecture. We have tested MDRNNs on two image segmentation tasks, and found that it was more robust to input warping than a state-of-the-art digit recognition algorithm.

References

[1] J.S. Bridle. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In F.Fogleman Soulie and J.Herault, editors, *Neurocomputing: Algorithms, Architectures and Applications*, pages 227–236. Springer-Verlag, 1990.

[2] F. Gers, N. Schraudolph, and J. Schmidhuber. Learning precise timing with LSTM recurrent networks. *Journal of Machine Learning Research*, 3:115–143, 2002.

[3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the International Conference on Machine Learning, ICML 2006*, Pittsburgh, USA, 2006.

[4] A. Graves, S. Fernández, and J. Schmidhuber. Bidirectional LSTM networks for improved phoneme classification and recognition. In *Proceedings of the 2005 International Conference on Artificial Neural Networks*, Warsaw, Poland, 2005.

[5] A. Graves and J. Schmidhuber. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks*, 18(5-6):602–610, 2005.

[6] S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. In S. C. Kremer and J. F. Kolen, editors, *A Field Guide to Dynamical Recurrent Neural Networks*. IEEE Press, 2001.

[7] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997.

[8] F. Hülsken, F. Wallhoff, and G. Rigoll. Facial expression recognition with pseudo-3d hidden markov models. In *Proceedings of the 23rd DAGM-Symposium on Pattern Recognition*, pages 291–297, London, UK, 2001. Springer-Verlag.

[9] J. Jiten, B. Mérialdo, and B. Huet. Multi-dimensional dependency-tree hidden Markov models. In *ICASSP 2006, 31st IEEE International Conference on Acoustics, Speech, and Signal Processing, May 14-19, 2006, Toulouse, France*, May 2006.

[10] D. Joshi, J. Li, and J.Z. Wang. Parameter estimation of multi-dimensional hidden markov models: A scalable approach. pages III: 149–152, 2005.

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998.

[12] J. (Stanford Univ) Li, A. Najmi, and R. M. Gray. Image classification by a two-dimensional hidden markov model. *IEEE Transactions on Signal Processing*, 48(2):517–533, 2000.

[13] M. Liwicki, A. Graves, H. Bunke, and J. Schmidhuber. A novel approach to on-line handwriting recognition based on bidirectional long short-term memory networks. In *Proceedings of the 9th International Conference on Document Analysis and Recognition, ICDAR 2007*, Curitiba, Brazil, 2007.

[14] M. Schuster and K. K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45:2673–2681, November 1997.

[15] P. Y. Simard, D. Steinkraus, and J. C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *ICDAR '03: Proceedings of the Seventh International Conference on Document Analysis and Recognition*, page 958, Washington, DC, USA, 2003. IEEE Computer Society.

[16] R. J. Williams and D. Zipser. Gradient-based learning algorithms for recurrent networks and their computational complexity. In Y. Chauvin and D. E. Rumelhart, editors, *Back-propagation: Theory, Architectures and Applications*, pages 433–486. Lawrence Erlbaum Publishers, Hillsdale, N.J., 1995.